

إضافة شهادات المدرب – الأكواد الكاملة

هذه الإضافة تحفظ شهادات المدرب في قاعدة البيانات، وتحفظ الملفات داخل `wwwroot/uploads/trainer-certificates`، ولا تستخدم `localStorage` كمصدر أساسي. نَقْذِي الأكواد بالترتيب. لا تحذف الأكواد الحالية في الملفات؛ أضيفي المقاطع في الأماكن المحددة.

1) Backend: المودل الجديد

أنشئي ملفاً جديداً:

Plain Text

Nafadh_Backend/Models/NFD_TrainerCertificate.cs

وضعي فيه:

C#

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Nafadh_Backend.Models
{
    public class NFD_TrainerCertificate
    {
        [Key]
        public int TrainerCertificateId { get; set; }

        [Required]
        public int TrainerId { get; set; }

        [Required, MaxLength(200)]
        public string CertificateName { get; set; } = string.Empty;

        [MaxLength(200)]
        public string? Issuer { get; set; }

        public DateTime? IssueDate { get; set; }
        public DateTime? ExpiryDate { get; set; }

        [Required, MaxLength(500)]
        public string FileUrl { get; set; } = string.Empty;

        [MaxLength(255)]
        public string? OriginalFileName { get; set; }
    }
}
```

```

        [MaxLength(100)]
        public string? ContentType { get; set; }

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        [ForeignKey(nameof(TrainerId))]
        public NFD_Trainer Trainer { get; set; } = null!;
    }
}

```

2) Backend: إضافة العلاقة داخل NFD_Trainer.cs

افتحي:

Plain Text

Nafadh_Backend/Models/NFD_Trainer.cs

بعد:

C#

```

public ICollection<NFD_Feedback> Feedbacks { get; set; } = new List<NFD_Feedback>();

```

أضيفي:

C#

```

public ICollection<NFD_TrainerCertificate> Certificates { get; set; }
    = new List<NFD_TrainerCertificate>();

```

3) Backend: DTOs

أنشئي الملف:

Plain Text

Nafadh_Backend/DTOs/TrainerCertificateDTO.cs

C#

```

using Microsoft.AspNetCore.Http;
using System.ComponentModel.DataAnnotations;

```

```

namespace Nafadh_Backend.DTOs
{
    public class TrainerCertificateDto
    {
        public int TrainerCertificateId { get; set; }
        public int TrainerId { get; set; }
        public string CertificateName { get; set; } = string.Empty;
        public string? Issuer { get; set; }
        public DateTime? IssueDate { get; set; }
        public DateTime? ExpiryDate { get; set; }
        public string FileUrl { get; set; } = string.Empty;
        public string? OriginalFileName { get; set; }
        public string? ContentType { get; set; }
        public DateTime CreatedAt { get; set; }
    }

    public class CreateTrainerCertificateDto
    {
        [Required, MaxLength(200)]
        public string CertificateName { get; set; } = string.Empty;

        [MaxLength(200)]
        public string? Issuer { get; set; }

        public DateTime? IssueDate { get; set; }
        public DateTime? ExpiryDate { get; set; }

        [Required]
        public IFormFile File { get; set; } = null!;
    }
}

```

4) Backend: Repository interface

أنشئي:

Plain Text

Nafadh_Backend/Repositories/ITrainerCertificateRepository.cs

C#

```

using Nafadh_Backend.Models;

namespace Nafadh_Backend.Repositories

```

```

{
    public interface ITrainerCertificateRepository
    {
        Task<List<NFD_TrainerCertificate>> GetByTrainerIdAsync(int trainerId);
        Task<NFD_TrainerCertificate?> GetByIdAsync(int id);
        Task AddAsync(NFD_TrainerCertificate certificate);
        Task<bool> DeleteAsync(int id);
    }
}

```

5) Backend: Repository implementation

أنشئي:

Plain Text

Nafadh_Backend/Repositories/TrainerCertificateRepository.cs

C#

```

using Microsoft.EntityFrameworkCore;
using Nafadh_Backend.Models;

namespace Nafadh_Backend.Repositories
{
    public class TrainerCertificateRepository : ITrainerCertificateRepository
    {
        private readonly Nafadhcontext _context;

        public TrainerCertificateRepository(Nafadhcontext context)
        {
            _context = context;
        }

        public async Task<List<NFD_TrainerCertificate>> GetByTrainerIdAsync(int
        {
            return await _context.NFD_TrainerCertificates
                .Where(x => x.TrainerId == trainerId)
                .OrderByDescending(x => x.CreatedAt)
                .ToListAsync();
        }

        public async Task<NFD_TrainerCertificate?> GetByIdAsync(int id)
        {
            return await _context.NFD_TrainerCertificates
                .FirstOrDefaultAsync(x => x.TrainerCertificateId == id);
        }
    }
}

```

```

    }

    public async Task AddAsync(NFD_TrainerCertificate certificate)
    {
        await _context.NFD_TrainerCertificates.AddAsync(certificate);
        await _context.SaveChangesAsync();
    }

    public async Task<bool> DeleteAsync(int id)
    {
        var certificate = await GetByIdAsync(id);
        if (certificate == null)
            return false;

        _context.NFD_TrainerCertificates.Remove(certificate);
        await _context.SaveChangesAsync();
        return true;
    }
}
}

```

6) Backend: Service interface

أنشئي:

Plain Text

Nafadh_Backend/Services/ITrainerCertificateService.cs

C#

```

using Microsoft.AspNetCore.Http;
using Nafadh_Backend.DTOS;

namespace Nafadh_Backend.Services
{
    public interface ITrainerCertificateService
    {
        Task<List<TrainerCertificateDto>> GetByTrainerIdAsync(int trainerId);
        Task<TrainerCertificateDto?> AddAsync(
            int trainerId,
            string certificateName,
            string? issuer,
            DateTime? issueDate,
            DateTime? expiryDate,
            IFormFile file);
    }
}

```

```
        Task<bool> DeleteAsync(int id);  
    }  
}
```

7) Backend: Service implementation

أنشئي:

Plain Text

Nafadh_Backend/Services/TrainerCertificateService.cs

C#

```
using Microsoft.AspNetCore.Http;  
using Nafadh_Backend.DTOs;  
using Nafadh_Backend.Models;  
using Nafadh_Backend.Repositories;  
  
namespace Nafadh_Backend.Services  
{  
    public class TrainerCertificateService : ITrainerCertificateService  
    {  
        private readonly ITrainerCertificateRepository _repository;  
        private readonly Nafadhcontext _context;  
        private readonly IWebHostEnvironment _environment;  
  
        private static readonly HashSet<string> AllowedExtensions =  
            new(StringComparer.OrdinalIgnoreCase)  
            {  
                ".pdf", ".jpg", ".jpeg", ".png"  
            };  
  
        private const long MaxFileSize = 10 * 1024 * 1024;  
  
        public TrainerCertificateService(  
            ITrainerCertificateRepository repository,  
            Nafadhcontext context,  
            IWebHostEnvironment environment)  
        {  
            _repository = repository;  
            _context = context;  
            _environment = environment;  
        }  
  
        public async Task<List<TrainerCertificateDto>> GetByTrainerIdAsync(int
```

```

{
    var items = await _repository.GetByTrainerIdAsync(trainerId);
    return items.Select(ToDto).ToList();
}

public async Task<TrainerCertificateDto?> AddAsync(
    int trainerId,
    string certificateName,
    string? issuer,
    DateTime? issueDate,
    DateTime? expiryDate,
    IFormFile file)
{
    var trainerExists = await _context.NFD_Trainers
        .AnyAsync(x => x.TrainerId == trainerId);

    if (!trainerExists)
        return null;

    if (file == null || file.Length == 0)
        throw new ArgumentException("الملف فارغ.");

    if (file.Length > MaxFileSize)
        throw new ArgumentException("حجم الملف يجب ألا يتجاوز 10 MB.");

    var extension = Path.GetExtension(file.FileName);
    if (!AllowedExtensions.Contains(extension))
        throw new ArgumentException("نقط PNG أو JPEG أو JPG أو PDF المسموح:");

    if (string.IsNullOrEmpty(certificateName))
        throw new ArgumentException("اسم الشهادة مطلوب.");

    if (expiryDate.HasValue && issueDate.HasValue && expiryDate < issueDate)
        throw new ArgumentException("تاريخ الانتهاء لا يمكن أن يسبق تاريخ الإصدار.");

    var relativeFolder = Path.Combine("uploads", "trainer-certificates");
    var absoluteFolder = Path.Combine(
        _environment.WebRootPath ?? Path.Combine(_environment.ContentRootPath,
        relativeFolder));

    Directory.CreateDirectory(absoluteFolder);

    var storedFileName = $"{Guid.NewGuid():N}{extension.ToLowerInvariant}";
    var absoluteFilePath = Path.Combine(absoluteFolder, storedFileName);

    await using (var stream = new FileStream(absoluteFilePath, FileMode.Create))
    {
        await file.CopyToAsync(stream);
    }
}

```

```

    }

    var certificate = new NFD_TrainerCertificate
    {
        TrainerId = trainerId,
        CertificateName = certificateName.Trim(),
        Issuer = string.IsNullOrWhiteSpace(issuer) ? null : issuer.Trim(),
        IssueDate = issueDate,
        ExpiryDate = expiryDate,
        FileUrl = $"/{relativeFolder.Replace('\\', '/')}/{storedFileName}",
        OriginalFileName = Path.GetFileName(file.FileName),
        ContentType = file.ContentType,
        CreatedAt = DateTime.UtcNow
    };

    try
    {
        await _repository.AddAsync(certificate);
        return ToDto(certificate);
    }
    catch
    {
        if (File.Exists(absoluteFilePath))
            File.Delete(absoluteFilePath);
        throw;
    }
}

public async Task<bool> DeleteAsync(int id)
{
    var certificate = await _repository.GetByIdAsync(id);
    if (certificate == null)
        return false;

    var filePath = GetPhysicalPath(certificate.FileUrl);
    var deleted = await _repository.DeleteAsync(id);

    if (deleted && filePath != null && File.Exists(filePath))
        File.Delete(filePath);

    return deleted;
}

private string? GetPhysicalPath(string? url)
{
    if (string.IsNullOrWhiteSpace(url))
        return null;

```



```

        var cleanUrl = url.TrimStart('/').Replace('/', Path.DirectorySeparatorChar);
        return Path.Combine(
            _environment.WebRootPath ?? Path.Combine(_environment.ContentRootPath,
                cleanUrl);
    }

    private static TrainerCertificateDto ToDto(NFD_TrainerCertificate x)
    {
        return new TrainerCertificateDto
        {
            TrainerCertificateId = x.TrainerCertificateId,
            TrainerId = x.TrainerId,
            CertificateName = x.CertificateName,
            Issuer = x.Issuer,
            IssueDate = x.IssueDate,
            ExpiryDate = x.ExpiryDate,
            FileUrl = x.FileUrl,
            OriginalFileName = x.OriginalFileName,
            ContentType = x.ContentType,
            CreatedAt = x.CreatedAt
        };
    }
}

```

إذا ظهر خطأ في AnyAsync ، أضيفي في أعلى الملف:

#C

```
using Microsoft.EntityFrameworkCore;
```

8) Backend: Controller

أنشئي:

Plain Text

Nafadh_Backend/Controllers/TrainerCertificateController.cs

C#

```

using Microsoft.AspNetCore.Mvc;
using Nafadh_Backend.DTOS;
using Nafadh_Backend.Services;

```

```

namespace Nafadh_Backend.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class TrainerCertificateController : ControllerBase
    {
        private readonly ITrainerCertificateService _service;

        public TrainerCertificateController(ITrainerCertificateService service)
        {
            _service = service;
        }

        [HttpGet("trainer/{trainerId:int}")]
        public async Task<IActionResult> GetByTrainer(int trainerId)
        {
            var result = await _service.GetByTrainerIdAsync(trainerId);
            return Ok(result);
        }

        [HttpPost("trainer/{trainerId:int}")]
        [Consumes("multipart/form-data")]
        [RequestSizeLimit(10 * 1024 * 1024)]
        public async Task<IActionResult> Add(
            int trainerId,
            [FromForm] CreateTrainerCertificateDto dto)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            try
            {
                var result = await _service.AddAsync(
                    trainerId,
                    dto.CertificateName,
                    dto.Issuer,
                    dto.IssueDate,
                    dto.ExpiryDate,
                    dto.File);

                if (result == null)
                    return NotFound(new { message = "المدرّب غير موجود." });

                return Ok(result);
            }
            catch (ArgumentException ex)
            {
                return BadRequest(new { message = ex.Message });
            }
        }
    }
}

```

```

    }
}

[HttpDelete("{id:int}")]
public async Task<IActionResult> Delete(int id)
{
    var deleted = await _service.DeleteAsync(id);
    if (!deleted)
        return NotFound(new { message = "الشهادة غير موجودة." });

    return NoContent();
}
}
}

```

9) Backend: DbSet و Mapping إضافة

افتحي:

Plain Text

Nafadh_Backend/Nafadhcontext.cs

بعد هذا السطر:

C#

```
public DbSet<NFD_Certificate> NFD_Certificates { get; set; }
```

أضيفي:

C#

```
public DbSet<NFD_TrainerCertificate> NFD_TrainerCertificates { get; set; }
```

ثم داخل OnModelCreating ، بعد بلوك // ---- NFD_Trainer ---- ، أضيفي:

C#

```
// ---- NFD_TrainerCertificate ----
modelBuilder.Entity<NFD_TrainerCertificate>(entity =>
{
    entity.ToTable("NFD_TrainerCertificates");

    entity.HasKey(e => e.TrainerCertificateId);

```

```

entity.Property(e => e.CertificateName)
    .HasMaxLength(200)
    .IsRequired();

entity.Property(e => e.Issuer)
    .HasMaxLength(200);

entity.Property(e => e.FileUrl)
    .HasMaxLength(500)
    .IsRequired();

entity.Property(e => e.OriginalFileName)
    .HasMaxLength(255);

entity.Property(e => e.ContentType)
    .HasMaxLength(100);

entity.HasOne(e => e.Trainer)
    .WithMany(t => t.Certificates)
    .HasForeignKey(e => e.TrainerId)
    .OnDelete(DeleteBehavior.Cascade);
});

```

تأكدني أن أعلى الملف يحتوي على:

C#

```
using Nafadh_Backend.Models;
```

10) Backend: Repository و Service تسجيل

افتحي:

Plain Text

Nafadh_Backend/Program.cs

بعد تسجيلات الشهادات الحالية، مثلاً بعد:

C#

```
builder.Services.AddScoped<ICertificateService, CertificateService>();
```

أضيفي:

C#

```
builder.Services.AddScoped<ITrainerCertificateRepository, TrainerCertificateRepository>();  
builder.Services.AddScoped<ITrainerCertificateService, TrainerCertificateService>();
```

11) Backend: Static Files التأكد من

تأكد من أن Program.cs يحتوي على هذا السطر قبل : ()app.MapControllers

C#

```
app.UseStaticFiles();
```

إذا كان موجوداً فلا تضيفه مرة ثانية.

12) إنشاء قاعدة البيانات

من داخل مجلد الـ Backend شغلي:

Bash

```
dotnet ef migrations add AddTrainerCertificates  
dotnet ef database update
```

إذا لم يكن dotnet ef مثبتاً:

Bash

```
dotnet tool install --global dotnet-ef
```

أو يمكن إنشاء الجدول مباشرة في SQL Server:

SQL

```
CREATE TABLE NFD_TrainerCertificates  
(  
    TrainerCertificateId INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    TrainerId INT NOT NULL,  
    CertificateName NVARCHAR(200) NOT NULL,  
    Issuer NVARCHAR(200) NULL,  
    IssueDate DATETIME2 NULL,  
    ExpiryDate DATETIME2 NULL,  
    FileUrl NVARCHAR(500) NOT NULL,  
    OriginalFileName NVARCHAR(255) NULL,
```

```

    ContentType NVARCHAR(100) NULL,
    CreatedAt DATETIME2 NOT NULL CONSTRAINT DF_TrainerCertificates_CreatedAt DE
    CONSTRAINT FK_TrainerCertificates_Trainers
        FOREIGN KEY (TrainerId)
        REFERENCES NFD_Trainers(TrainerId)
        ON DELETE CASCADE
);

CREATE INDEX IX_TrainerCertificates_TrainerId
ON NFD_TrainerCertificates(TrainerId);

```

استخدم Migration أو SQL، وليس الاثنين معاً على نفس قاعدة البيانات.

Frontend

13) Frontend: DTO

افتحي:

Plain Text

Nafadh_Frontend/src/app/core/models/dtos.ts

بعد أي interface خاص بالمدرّب، أضيفي:

Plain Text

```

export interface TrainerCertificateDto {
  trainerCertificateId: number;
  trainerId: number;
  certificateName: string;
  issuer?: string | null;
  issueDate?: string | null;
  expiryDate?: string | null;
  fileUrl: string;
  originalFileName?: string | null;
  contentType?: string | null;
  createdAt: string;
}

```

14) Frontend: استيراد DTO في trainer-api.ts

افتحي:

Plain Text

Nafadh_Frontend/src/app/portals/trainer/services/trainer-api.ts

في قائمة الاستيراد من dtos أضيفي:

Plain Text

TrainerCertificateDto,

مثلاً بعد:

Plain Text

TrainerDto,

ثم قبل القوس الأخير للكلاس TrainerApi ، أي قبل آخر { في الملف، أضيفي:

Plain Text

```
// =====  
// TRAINER CERTIFICATES  
// =====  
  
getTrainerCertificates(  
  trainerId: number  
): Observable<TrainerCertificateDto[]> {  
  return this.http.get<TrainerCertificateDto[]>(  
    `${this.base}/TrainerCertificate/trainer/${trainerId}`  
  );  
}  
  
uploadTrainerCertificate(  
  trainerId: number,  
  certificateName: string,  
  issuer: string,  
  issueDate: string,  
  expiryDate: string,  
  file: File  
): Observable<TrainerCertificateDto> {  
  const formData = new FormData();  
  
  formData.append('CertificateName', certificateName);  
  formData.append('Issuer', issuer || '');  
  formData.append('IssueDate', issueDate || '');  
  formData.append('ExpiryDate', expiryDate || '');
```

```

formData.append('File', file);

return this.http.post<TrainerCertificateDto>(
  `${this.base}/TrainerCertificate/trainer/${trainerId}`,
  formData
);
}

deleteTrainerCertificate(
  certificateId: number
): Observable<void> {
  return this.http.delete<void>(
    `${this.base}/TrainerCertificate/${certificateId}`
  );
}

```

15) Frontend: متغيرات الشهادات في profile.ts

افتحي:

Plain Text

Nafadh_Frontend/src/app/portals/trainer/pages/profile/profile.ts

عدّلي import الـ DTO الحالي ليصبح مثلاً:

Plain Text

```

import {
  TrainerBatchDto,
  TrainerCertificateDto,
  TrainerDto
} from '../../../core/models/dtos';

```

بعد المتغير:

Plain Text

```

trainerBatches = signal<TrainerBatchDto[]>([]);

```

أضيفي:

Plain Text


```
// =====
// TRAINER CERTIFICATES
// =====

trainerCertificates = signal<TrainerCertificateDto[]>([]);

certificateName = '';
certificateIssuer = '';
certificateIssueDate = '';
certificateExpiryDate = '';
selectedCertificateFile: File | null = null;
isUploadingCertificate = signal(false);

certificateFileInput: HTMLInputElement | null = null;
```

16) Frontend: تحميل الشهادات

داخل `loadTrainer()` ، في `next: (data) => { ... }` ، بعد:

Plain Text

```
this.loadTrainerBatches(data.trainerId);
```

أضيفي:

Plain Text

```
this.loadTrainerCertificates(data.trainerId);
```

ثم أضيفي هذه الدوال داخل كلاس `TrainerProfile` ، قبل القوس الأخير `{`:

Plain Text

```
loadTrainerCertificates(trainerId: number): void {
  this.api.getTrainerCertificates(trainerId).subscribe({
    next: (items) => {
      this.trainerCertificates.set(items ?? []);
    },
    error: (error) => {
      console.error('تعذر تحميل شهادات المدرب', error);
      this.trainerCertificates.set([]);
    }
  });
}
```

```

onCertificateFileSelected(event: Event): void {
    const input = event.target as HTMLInputElement;
    this.selectedCertificateFile = input.files?.[0] ?? null;
}

uploadCertificate(): void {
    const currentTrainer = this.trainer();

    if (!currentTrainer?.trainerId) {
        this.showError('تعذر تحديد المدرب الحالي. ');
        return;
    }

    if (!this.certificateName.trim()) {
        this.showError('اكتب اسم الشهادة أولاً. ');
        return;
    }

    if (!this.selectedCertificateFile) {
        this.showError('اختر ملف الشهادة أولاً. ');
        return;
    }

    this.isUploadingCertificate.set(true);

    this.api.uploadTrainerCertificate(
        currentTrainer.trainerId,
        this.certificateName.trim(),
        this.certificateIssuer.trim(),
        this.certificateIssueDate,
        this.certificateExpiryDate,
        this.selectedCertificateFile
    ).subscribe({
        next: (certificate) => {
            this.trainerCertificates.update(items => [certificate, ...items]);
            this.resetCertificateForm();
            this.isUploadingCertificate.set(false);
            this.showSuccess('تم رفع الشهادة وحفظها بنجاح. ');
        },
        error: (error) => {
            console.error('تعذر رفع الشهادة:', error);
            this.isUploadingCertificate.set(false);
            this.showError(error?.error?.message ?? 'تعذر رفع الشهادة. ');
        }
    });
}

deleteCertificate(certificate: TrainerCertificateDto): void {

```

```

const confirmed = window.confirm(
  `هل تريد حذف شهادة "${certificate.certificateName}"؟`
);

if (!confirmed) return;

this.api.deleteTrainerCertificate(
  certificate.trainerCertificateId
).subscribe({
  next: () => {
    this.trainerCertificates.update(items =>
      items.filter(x =>
        x.trainerCertificateId !== certificate.trainerCertificateId
      )
    );
    this.showSuccess('تم حذف الشهادة.');
```

},

```

    error: (error) => {
      console.error('تعذر حذف الشهادة:', error);
      this.showError('تعذر حذف الشهادة.');
```

}

```

  });
}

private resetCertificateForm(): void {
  this.certificateName = '';
  this.certificateIssuer = '';
  this.certificateIssueDate = '';
  this.certificateExpiryDate = '';
  this.selectedCertificateFile = null;

  if (this.certificateFileInput) {
    this.certificateFileInput.value = '';
  }
}

getCertificateFileUrl(fileUrl: string): string {
  if (!fileUrl) return '#';

  if (fileUrl.startsWith('http://') || fileUrl.startsWith('https://')) {
    return fileUrl;
  }

  const backendBaseUrl = environment.apiBaseUrl.replace(/\/api\/?$/, '');
  return `${backendBaseUrl}${fileUrl.startsWith('/') ? fileUrl : `/${fileUrl}`}`
}

```

17) Frontend: HTML section

افتحي:

Plain Text

```
Nafadh_Frontend/src/app/portals/trainer/pages/profile/profile.html
```

داخل:

HTML

```
@if (!loading() && trainer()); as t) {
```

أضيفي المقطع التالي بعد إغلاق قسم المعلومات الشخصية، أي بعد:

HTML

```
</section>
```

الخاص بـ profile-card وقبل قسم summary-card :

HTML

```
<section class="certificates-card">
  <div class="certificates-header">
    <div>
      <span class="summary-kicker">الملف المهني</span>
      <h2 class="summary-title">الشهادات المهنية</h2>
      <p class="summary-sub">
        أضيفي شهادتك المهنية ليتم حفظها في النظام.
      </p>
    </div>

    <div class="certificate-header-icon">
      <i class="fa-solid fa-certificate"></i>
    </div>
  </div>

  <div class="certificate-upload-form">
    <div class="certificate-field">
      <label>اسم الشهادة *</label>
      <input
        type="text"
        [(ngModel)]="certificateName"
        placeholder="مثال: Microsoft Azure Fundamentals"
      >
    </div>
  </div>
</section>
```

```

    />
</div>

<div class="certificate-field">
  <label>الجهة المصدرة</label>
  <input
    type="text"
    [(ngModel)]="certificateIssuer"
    placeholder="مثال: Microsoft"
  />
</div>

<div class="certificate-field">
  <label>تاريخ الإصدار</label>
  <input
    type="date"
    [(ngModel)]="certificateIssueDate"
  />
</div>

<div class="certificate-field">
  <label>تاريخ الانتهاء</label>
  <input
    type="date"
    [(ngModel)]="certificateExpiryDate"
  />
</div>

<div class="certificate-field full-width">
  <label>ملف الشهادة *</label>
  <input
    #certificateFile
    type="file"
    accept=".pdf, .jpg, .jpeg, .png"
    (change)="onCertificateFileSelected($event)"
  />
  <small>PDF أو JPG أو PNG – الحد الأقصى 10 MB</small>
</div>

<button
  type="button"
  class="certificate-upload-btn"
  [disabled]="isUploadingCertificate()"
  (click)="uploadCertificate()"
>
  @if (isUploadingCertificate()) {
    جاري الرفع...
  } @else {

```

```

        <i class="fa-solid fa-upload"></i>
        إضافة الشهادة
    }
</button>
</div>

<div class="certificates-list">
    @if (trainerCertificates().length === 0) {
        <div class="certificates-empty">
            لا توجد شهادات مضافة حتى الآن.
        </div>
    } @else {
        @for (certificate of trainerCertificates(); track certificate.trainerCert
        <article class="certificate-item">
            <div class="certificate-item-icon">
                <i class="fa-solid fa-award"></i>
            </div>

            <div class="certificate-item-content">
                <h3>{{ certificate.certificateName }}</h3>

                @if (certificate.issuer) {
                    <span>الجهة المصدرة: {{ certificate.issuer }}</span>
                }

                <div class="certificate-dates">
                    @if (certificate.issueDate) {
                        <span>الإصدار: {{ certificate.issueDate | date:'yyyy/MM/dd' }}</s
                    }
                    @if (certificate.expiryDate) {
                        <span>الانتهاء: {{ certificate.expiryDate | date:'yyyy/MM/dd' }}</s
                    }
                </div>
            </div>

            <div class="certificate-item-actions">
                <a
                    class="certificate-view-btn"
                    [href]="getCertificateFileUrl(certificate.fileUrl)"
                    target="_blank"
                    rel="noopener"
                >
                    عرض الملف
                </a>

                <button
                    type="button"
                    class="certificate-delete-btn"

```

```

        (click)="deleteCertificate(certificate)"
      >
        حذف
      </button>
    </div>
  </article>
}
}
</div>
</section>

```

إضافي import سيعمل بدون date pipe لذلك ، profile.ts موجود أصلاً في CommonModule

18) Frontend: CSS

افتحي:

Plain Text

Nafadh_Frontend/src/app/portals/trainer/pages/profile/profile.scss

في نهاية الملف أضيفي:

Plain Text

```

.certificates-card {
  margin-top: 24px;
  padding: 24px;
  background: #ffffff;
  border: 1px solid #e5eef2;
  border-radius: 18px;
  box-shadow: 0 8px 24px rgba(15, 77, 92, 0.05);
}

.certificates-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 22px;
}

.certificate-header-icon,
.certificate-item-icon {
  display: grid;
  place-items: center;
}

```

```
    color: #087f95;
    background: #e8f8fa;
    border-radius: 14px;
}

.certificate-header-icon {
    width: 48px;
    height: 48px;
    font-size: 21px;
}

.certificate-upload-form {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 14px;
    padding: 18px;
    background: #f7fbfc;
    border: 1px solid #e6f0f2;
    border-radius: 14px;
}

.certificate-field {
    display: flex;
    flex-direction: column;
    gap: 7px;
}

.certificate-field.full-width {
    grid-column: 1 / -1;
}

.certificate-field label {
    color: #294750;
    font-size: 13px;
    font-weight: 700;
}

.certificate-field input {
    min-height: 42px;
    padding: 9px 12px;
    border: 1px solid #d7e5e8;
    border-radius: 9px;
    background: #fff;
    color: #203a41;
    font: inherit;
}

.certificate-field input:focus {
```



```
outline: none;
border-color: #0b91a5;
box-shadow: 0 0 0 3px rgba(11, 145, 165, 0.12);
}

.certificate-field small {
  color: #71878d;
  font-size: 11px;
}

.certificate-upload-btn {
  grid-column: 1 / -1;
  justify-self: start;
  min-height: 42px;
  padding: 0 20px;
  border: 0;
  border-radius: 9px;
  background: #087f95;
  color: #fff;
  cursor: pointer;
  font: inherit;
  font-weight: 700;
}

.certificate-upload-btn:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.certificates-list {
  display: flex;
  flex-direction: column;
  gap: 12px;
  margin-top: 18px;
}

.certificates-empty {
  padding: 24px;
  color: #71878d;
  text-align: center;
  border: 1px dashed #cbdfef;
  border-radius: 12px;
}

.certificate-item {
  display: flex;
  align-items: center;
  gap: 14px;
```

```
padding: 15px;
border: 1px solid #e5eef0;
border-radius: 13px;
background: #fff;
}

.certificate-item-icon {
  flex: 0 0 42px;
  width: 42px;
  height: 42px;
}

.certificate-item-content {
  flex: 1;
  min-width: 0;
}

.certificate-item-content h3 {
  margin: 0 0 5px;
  color: #21424b;
  font-size: 15px;
}

.certificate-item-content span {
  color: #71878d;
  font-size: 12px;
}

.certificate-dates {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin-top: 5px;
}

.certificate-item-actions {
  display: flex;
  align-items: center;
  gap: 8px;
}

.certificate-view-btn,
.certificate-delete-btn {
  padding: 8px 11px;
  border-radius: 8px;
  font: inherit;
  font-size: 12px;
  cursor: pointer;
}
```

```
text-decoration: none;
}

.certificate-view-btn {
border: 1px solid #b7dce3;
color: #087f95;
background: #f3fbfc;
}

.certificate-delete-btn {
border: 1px solid #f1c7c7;
color: #bd4c4c;
background: #fff7f7;
}

@media (max-width: 700px) {
.certificate-upload-form {
grid-template-columns: 1fr;
}

.certificate-field.full-width,
.certificate-upload-btn {
grid-column: auto;
}

.certificate-item {
align-items: flex-start;
flex-wrap: wrap;
}

.certificate-item-actions {
width: 100%;
padding-right: 56px;
}
}
```

(19) الفحص والتشغيل

من مجلد الـBackend:

Bash

```
dotnet build
```

ومن مجلد الـFrontend:

Bash

```
npm install
ng build
```

ثم شغلي الـ Backend والـ Frontend وجربي:

1. تسجيل الدخول كمدرّب.
2. فتح صفحة الملف الشخصي.
3. كتابة اسم شهادة.
4. اختيار PDF أو صورة.
5. الضغط على «إضافة الشهادة».
6. فتح «عرض الملف».
7. تحديث الصفحة والتأكد أن الشهادة ما زالت موجودة.
8. تجربة الحذف.

النتيجة Endpoints

Plain Text

```
GET      /api/TrainerCertificate/trainer/{trainerId}
POST     /api/TrainerCertificate/trainer/{trainerId}
DELETE   /api/TrainerCertificate/{id}
```

ملاحظات مهمة

- لا تعدّلي NFD_Certificate.cs ؛ هذا خاص بشهادات المتدربين.
- لا تضعي ملفات الشهادات في localStorage .
- لا تغيّري ProfileImageUrl ؛ صورة المدرّب لها وظيفة مختلفة.
- إذا كان السيرفر يعمل خلف IIS أو Docker، تأكّدي أن مجلد `wwwroot/uploads/trainer-certificates` لديه صلاحية كتابة.
- في الإنتاج يفضل نقل الملفات إلى تخزين خارجي، لكن هذا الحل مناسب لبيئة المشروع الحالية مثل طريقة رفع صورة المدرّب ومواد التدريب.